

E-Commerce Frontend Architecture (HTML, CSS, JS + LocalStorage)

مقدمة

حيث Backend كامل لبيع المنتجات (أي نوع منتجات)، لكن بدون E-Commerce المشروع عبارة عن موقع لتخزين LocalStorage فقط، مع الاعتماد على HTML + CSS + JavaScript يتم تنفيذ كل شيء باستخدام البيانات مثل المستخدمين، المنتجات، السلة، والطلبات.

الهدف من المشروع هو بناء نظام يحاكي مواقع التجارة الإلكترونية الحقيقية من حيث الوظائف الأساسية. (Frontend) ولكن بطريقة مبسطة تعمل بالكامل على الواجهة الأمامية.

فكرة النظام (Architecture Overview)

يتكون المشروع من 3 طبقات رئيسية:

1. واجهة المستخدم (UI Layer)

- بناء الصفحات: HTML
- التصميم والتنسيق: CSS

2. منطق التطبيق (Logic Layer)

- للتحكم في JavaScript:
 - تسجيل الدخول
 - إدارة السلة
 - عرض المنتجات
 - تنفيذ العمليات المختلفة

3. طبقة البيانات (Data Layer)

- لتخزين LocalStorage:
 - المستخدمين (Users)
 - المنتجات (Products)
 - السلة (Cart)
 - الطلبات (Orders)

هيكل المشروع (Project Structure)

/ecommerce

|

— index.html (Home)

— product.html (Product Details)

```
├── cart.html      (Cart)
├── checkout.html  (Checkout)
├── login.html     (Login)
├── register.html  (Register)
├── profile.html   (User Profile)
|
├── /css
|   └── style.css
|
├── /js
|   ├── main.js
|   ├── auth.js
|   ├── products.js
|   ├── cart.js
|   ├── checkout.js
|   └── storage.js
|
└── /data (اختياري)
    └── products.js
```

المميزات الأساسية (Core Features)

1. نظام تسجيل الدخول (Authentication)

- تسجيل حساب (Register)
- تسجيل دخول (Login)
- تسجيل خروج (Logout)

يتم تخزين:

- users
- currentUser

2. الصفحة الرئيسية (Home Page)

- عرض جميع المنتجات
- إمكانية البحث
- عرض الفئات (اختياري)

3. المنتجات (Products)

- عرض المنتجات
- صفحة تفاصيل لكل منتج

كل منتج يحتوي على:

- id
- name
- price
- image
- description
- category

4. سلة المشتريات (Cart)

- إضافة منتج
- حذف منتج
- تعديل الكمية
- حساب السعر الإجمالي

5. الدفع (Checkout - Fake)

- إدخال بيانات:
 - الاسم
 - العنوان
 - رقم الهاتف
- اختيار طريقة الدفع (كاش / فيزا وهمية)

وتخزينه Order يتم إنشاء

6. حساب المستخدم (Profile)

- عرض بيانات المستخدم
- عرض الطلبات السابقة

7. البحث والتصفية

- البحث بالاسم
- الفلترة حسب الفئة

مميزات إضافية (Optional Features)

لرفع مستوى المشروع:

- Wishlist
- تقييم المنتجات
- Dark Mode
- Pagination
- Notifications (Toast)

LocalStorage إدارة البيانات باستخدام

Helper Functions

```
function getData(key) {  
  
  return JSON.parse(localStorage.getItem(key)) || [];  
  
}
```

```
function setData(key, data) {  
  
  localStorage.setItem(key, JSON.stringify(data));  
  
}
```

مثال: إضافة منتج للسلة

```
function addToCart(productId) {  
  
  let cart = getData("cart");  
  
  
  
  let item = cart.find(p => p.productId === productId);  
  
  
  
  if (item) {  
  
    item.quantity++;  
  
  } else {  
  
    cart.push({ productId, quantity: 1 });  
  
  }  
  
}
```

```
setData("cart", cart);
```

```
}
```

حماية الصفحات (Auth Guard)

```
let user = JSON.parse(localStorage.getItem("currentUser"));
```

```
if (!user) {
```

```
  window.location.href = "login.html";
```

```
}
```

تجربة المستخدم (User Flow)

- المستخدم يسجل حساب
- يعمل تسجيل دخول
- يتصفح المنتجات
- يضيف منتجات إلى السلة
- يذهب إلى الدفع
- يتم حفظ الطلب

الصفحات الأساسية (Pages)

- Home
- Product Details
- Cart
- Checkout
- Login / Register
- Profile

ملاحظات مهمة

- (Backend لأنه بدون) لا يوجد أمان حقيقي
- البيانات يمكن تعديلها من المتصفح
- محدود في الحجم LocalStorage
- (فقط Simulation) الدفع غير حقيقي

أفضل الممارسات (Best Practices)

- Modules تقسيم الكود إلى:
 - auth.js
 - cart.js
 - products.js
 - storage.js

- (Clean Code) كتابة كود منظم
- فصل المنطق عن الواجهة
- (Reusable Functions) إعادة استخدام الدوال

الخلاصة

فقط Frontend حقيقي لكن باستخدام E-Commerce هذا المشروع يمثل محاكاة كاملة لموقع. رغم بساطته من ناحية التقنية، إلا أنه يغطي جميع المفاهيم الأساسية مثل:

- إدارة المستخدمين
- إدارة المنتجات
- التعامل مع السلة
- تنفيذ عمليات الشراء

كمشروع عملي احترافي CV وهو مشروع قوي جدًا لو تم تنفيذه بشكل منظم، ويمكن إضافته في الـ

الخطوة الجاية:

نقدر نحول الكلام ده لكود فعلي ونبدأ بـ

- Authentication
- Products بعده
- Cart بعده
- Checkout بعده